Mobile Application Development

Android Studio with Java and XML

Comprehensive Code Snippet Notes
Basic level - for MCA students

Unit	Topics
1	Getting started: project structure, findViewById, click events
2	Form widgets (TextView, EditText, Button, CheckBox, RadioButton, Switch, RatingBar, SeekBar...)
3	Text fields, input types and validation
4	Layouts (Linear, Relative, Constraint, Frame, Table, Grid)
5	Adapters, Spinner, ListView (simple + custom)
6	AlertDialog, Toast, Popup menu and PopupWindow
7	Fragments
8	IME (keyboard) options, DatePicker and TimePicker
9	SharedPreferences
10	Explicit and Implicit Intents
11	Quick revision cheat sheet and common viva questions

Note: All examples use **AppCompatActivity**, **androidx** libraries and **findViewById()** (no ViewBinding) to keep things simple. Package name used throughout: `com.example.myapp`. Replace ids and names to suit your project.

Unit 1: Getting Started

1.1 Project structure (important files)

Path	Purpose
app/src/main/java/.../MainActivity.java	Java code (logic) of the screen
app/src/main/res/layout/activity_main.xml	XML layout (UI design) of the screen
app/src/main/res/values/strings.xml	All text strings (supports localisation)
app/src/main/res/values/colors.xml	Colour resources
app/src/main/res/drawable/	Images, shapes, vector icons
app/src/main/res/menu/	Menu XML files
app/src/main/AndroidManifest.xml	Declares activities, permissions, app icon, launcher activity
app/build.gradle	Dependencies, SDK versions

1.2 Basic Activity skeleton

MainActivity.java

```
package com.example.myapp;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);   // link XML to this class
    }
}
```

1.3 Connecting XML view to Java and handling click

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:id="@+id/tvHello"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello" />

    <Button
        android:id="@+id/btnClick"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Click Me" />
</LinearLayout>
```

MainActivity.java - three ways to handle a click

```
TextView tvHello = findViewById(R.id.tvHello);
Button btnClick  = findViewById(R.id.btnClick);

// Way 1: anonymous inner class (most common)
btnClick.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        tvHello.setText("Button clicked!");
    }
});

// Way 2: lambda (Java 8+)
btnClick.setOnClickListener(v -> tvHello.setText("Clicked via lambda"));

// Way 3: android:onClick in XML  ->  android:onClick="showMessage"
public void showMessage(View view) {
    Toast.makeText(this, "XML onClick", Toast.LENGTH_SHORT).show();
}
```

1.4 Common XML attributes

Attribute	Meaning
layout_width / layout_height	match_parent, wrap_content or a fixed value (e.g. 100dp)
padding / layout_margin	Space inside / outside a view
gravity / layout_gravity	Alignment of content inside the view / of view inside parent
textSize (sp), textColor, textStyle	Text appearance (use sp for text, dp for sizes)
background	Colour or drawable, e.g. @color/white or #FF5722
visibility	visible, invisible (keeps space), gone (no space)
hint	Placeholder text for EditText
layout_weight	Share remaining space in a LinearLayout

Unit 2: Form Widgets

Form widgets are the input/output controls that make up a screen: labels, text boxes, buttons, choices and sliders.

2.1 TextView (label)

XML

```
<TextView
    android:id="@+id/tvTitle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Registration Form"
    android:textSize="22sp"
    android:textStyle="bold"
    android:textColor="#1F3A5F" />
```

Java

```
TextView tvTitle = findViewById(R.id.tvTitle);
tvTitle.setText("New title");
tvTitle.setTextColor(Color.RED);          // import android.graphics.Color
tvTitle.setTextSize(20);
String s = tvTitle.getText().toString();
```

2.2 EditText (text field) and Button

XML

```
<EditText
    android:id="@+id/etName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter your name"
    android:inputType="textPersonName" />

<Button
    android:id="@+id/btnSubmit"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Submit" />
```

Java

```
EditText etName = findViewById(R.id.etName);
Button btnSubmit = findViewById(R.id.btnSubmit);

btnSubmit.setOnClickListener(v -> {
    String name = etName.getText().toString().trim();
    Toast.makeText(this, "Hello " + name, Toast.LENGTH_SHORT).show();
});
```

2.3 CheckBox (multiple selection)

XML

```
<CheckBox android:id="@+id/cbJava" android:layout_width="wrap_content"
    android:layout_height="wrap_content" android:text="Java" />
<CheckBox android:id="@+id/cbPython" android:layout_width="wrap_content"
    android:layout_height="wrap_content" android:text="Python" />
```

Java

```
CheckBox cbJava = findViewById(R.id.cbJava);
CheckBox cbPython = findViewById(R.id.cbPython);

// Read on button click
StringBuilder sb = new StringBuilder();
if (cbJava.isChecked())   sb.append("Java ");
if (cbPython.isChecked()) sb.append("Python ");
Toast.makeText(this, "Selected: " + sb, Toast.LENGTH_SHORT).show();

// Listen to changes
cbJava.setOnCheckedChangeListener((buttonView, isChecked) ->
        Toast.makeText(this, "Java: " + isChecked, Toast.LENGTH_SHORT).show());
```

2.4 RadioGroup and RadioButton (single selection)

XML

```
<RadioGroup
    android:id="@+id/rgGender"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="horizontal">

    <RadioButton android:id="@+id/rbMale" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Male" />
    <RadioButton android:id="@+id/rbFemale" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Female" />
</RadioGroup>
```

Java

```
RadioGroup rgGender = findViewById(R.id.rgGender);

int selectedId = rgGender.getCheckedRadioButtonId();   // -1 if nothing selected
if (selectedId == -1) {
    Toast.makeText(this, "Select gender", Toast.LENGTH_SHORT).show();
} else {
    RadioButton rb = findViewById(selectedId);
    String gender = rb.getText().toString();
}

// Listener
rgGender.setOnCheckedChangeListener((group, checkedId) -> {
    RadioButton rb = findViewById(checkedId);
    Toast.makeText(this, rb.getText(), Toast.LENGTH_SHORT).show();
});
```

2.5 Switch and ToggleButton

XML

```
<Switch
    android:id="@+id/swWifi"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Notifications" />

<ToggleButton
    android:id="@+id/tbMode"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:textOn="ON"
    android:textOff="OFF" />
```

Java

```
Switch swWifi = findViewById(R.id.swWifi);   // android.widget.Switch
swWifi.setOnCheckedChangeListener((btn, isChecked) ->
    Toast.makeText(this, isChecked ? "Enabled" : "Disabled",
                   Toast.LENGTH_SHORT).show());
// Tip: SwitchCompat / SwitchMaterial can be used from AppCompat/Material library.
```

2.6 RatingBar and SeekBar

XML

```
<RatingBar
    android:id="@+id/ratingBar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:numStars="5"
    android:stepSize="0.5" />

<SeekBar
    android:id="@+id/seekBar"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:max="100"
    android:progress="20" />
```

Java

```
RatingBar ratingBar = findViewById(R.id.ratingBar);
float stars = ratingBar.getRating();
ratingBar.setOnRatingBarChangeListener((bar, rating, fromUser) ->
        Toast.makeText(this, "Rating: " + rating, Toast.LENGTH_SHORT).show());

SeekBar seekBar = findViewById(R.id.seekBar);
seekBar.setOnSeekBarChangeListener(new SeekBar.OnSeekBarChangeListener() {
    @Override
    public void onProgressChanged(SeekBar sb, int progress, boolean fromUser) {
        tvValue.setText("Value: " + progress);
    }
    @Override public void onStartTrackingTouch(SeekBar sb) { }
    @Override public void onStopTrackingTouch(SeekBar sb) { }
});
```

2.7 ProgressBar and ImageView

XML

```
<ProgressBar
    android:id="@+id/progressBar"
    style="?android:attr/progressBarStyleHorizontal"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:max="100" />

<ImageView
    android:id="@+id/imgLogo"
    android:layout_width="100dp"
    android:layout_height="100dp"
    android:src="@drawable/ic_launcher_foreground"
    android:scaleType="centerCrop"
    android:contentDescription="App logo" />
```

Java

```
ProgressBar progressBar = findViewById(R.id.progressBar);
progressBar.setProgress(60);
progressBar.setVisibility(View.GONE);      // hide it

ImageView img = findViewById(R.id.imgLogo);
img.setImageResource(R.drawable.my_photo);
```

2.8 Complete mini example: Registration form (collect and display)

activity_registration.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <EditText android:id="@+id/etName" android:layout_width="match_parent"
        android:layout_height="wrap_content" android:hint="Full name" />

    <RadioGroup android:id="@+id/rgGender" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:orientation="horizontal">
        <RadioButton android:id="@+id/rbMale" android:layout_width="wrap_content"
            android:layout_height="wrap_content" android:text="Male" />
        <RadioButton android:id="@+id/rbFemale" android:layout_width="wrap_content"
            android:layout_height="wrap_content" android:text="Female" />
    </RadioGroup>

    <CheckBox android:id="@+id/cbTerms" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="I accept the terms" />

    <Button android:id="@+id/btnRegister" android:layout_width="match_parent"
        android:layout_height="wrap_content" android:text="Register" />

    <TextView android:id="@+id/tvResult" android:layout_width="match_parent"
        android:layout_height="wrap_content" android:paddingTop="12dp" />
</LinearLayout>
```

RegistrationActivity.java

```
public class RegistrationActivity extends AppCompatActivity {
    EditText etName; RadioGroup rgGender; CheckBox cbTerms;
    Button btnRegister; TextView tvResult;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_registration);

        etName = findViewById(R.id.etName);
        rgGender = findViewById(R.id.rgGender);
        cbTerms = findViewById(R.id.cbTerms);
        btnRegister = findViewById(R.id.btnRegister);
        tvResult = findViewById(R.id.tvResult);

        btnRegister.setOnClickListener(v -> {
            String name = etName.getText().toString().trim();
            int id = rgGender.getCheckedRadioButtonId();

            if (name.isEmpty())        { etName.setError("Name required"); return; }
            if (id == -1)              { toast("Select gender"); return; }
            if (!cbTerms.isChecked())  { toast("Accept terms first"); return; }

            String gender = ((RadioButton) findViewById(id)).getText().toString();
            tvResult.setText("Name: " + name + "\nGender: " + gender);
        });
    }
    private void toast(String m) { Toast.makeText(this, m, Toast.LENGTH_SHORT).show(); }
}
```

Unit 3: Text Fields and Validation

3.1 Common inputType values for EditText

inputType	Effect
text	Normal text keyboard
textPersonName	Capitalises names
textEmailAddress	Keyboard with @ and .
textPassword	Hides characters with dots
number / numberDecimal / numberSigned	Numeric keypad
phone	Phone keypad
textMultiLine	Multi-line (use with lines / minLines)
date / time	Date or time entry keyboards

XML: different kinds of text fields

```
<EditText android:id="@+id/etEmail" android:layout_width="match_parent"
    android:layout_height="wrap_content" android:hint="Email"
    android:inputType="textEmailAddress" />

<EditText android:id="@+id/etPassword" android:layout_width="match_parent"
    android:layout_height="wrap_content" android:hint="Password"
    android:inputType="textPassword" />

<EditText android:id="@+id/etPhone" android:layout_width="match_parent"
    android:layout_height="wrap_content" android:hint="Mobile number"
    android:inputType="phone" android:maxLength="10" />

<EditText android:id="@+id/etAge" android:layout_width="match_parent"
    android:layout_height="wrap_content" android:hint="Age"
    android:inputType="number" android:maxLength="3" />

<EditText android:id="@+id/etAddress" android:layout_width="match_parent"
    android:layout_height="wrap_content" android:hint="Address"
    android:inputType="textMultiLine" android:lines="3" android:gravity="top" />
```

3.2 TextInputLayout (Material design text field)

Add dependency `implementation 'com.google.android.material:material:1.11.0'` (version may differ).

XML

```
<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/tilEmail"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Email">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/etEmail"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textEmailAddress" />
</com.google.android.material.textfield.TextInputLayout>
```

Java

```
TextInputLayout tilEmail = findViewById(R.id.tilEmail);
TextInputEditText etEmail = findViewById(R.id.etEmail);

tilEmail.setError("Invalid email");   // shows red message under field
tilEmail.setError(null);              // clears the error
```

3.3 Validation on button click (most common in exams)

Java: reusable validation methods

```
private boolean isEmpty(EditText et, String msg) {
    if (et.getText().toString().trim().isEmpty()) {
        et.setError(msg);       // red icon + message
        et.requestFocus();      // move cursor to this field
        return true;
    }
    return false;
}

private boolean isValidEmail(String email) {
    return android.util.Patterns.EMAIL_ADDRESS.matcher(email).matches();
}

private boolean isValidPhone(String phone) {
    return phone.matches("[6-9][0-9]{9}");        // Indian 10-digit mobile
}

private boolean isStrongPassword(String pwd) {
    // min 8 chars, 1 digit, 1 lowercase, 1 uppercase, 1 special char
    return pwd.matches("^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[@#$%^&+=!]).{8,}$");
}
```

Java: validate all fields on Submit

```
btnSubmit.setOnClickListener(v -> {
    String name  = etName.getText().toString().trim();
    String email = etEmail.getText().toString().trim();
    String phone = etPhone.getText().toString().trim();
    String pwd   = etPassword.getText().toString();
    String cpwd  = etConfirm.getText().toString();

    if (name.isEmpty()) {
        etName.setError("Name is required");
        etName.requestFocus();
    } else if (!isValidEmail(email)) {
        etEmail.setError("Enter a valid email");
        etEmail.requestFocus();
    } else if (!isValidPhone(phone)) {
        etPhone.setError("Enter a valid 10-digit mobile number");
        etPhone.requestFocus();
    } else if (pwd.length() < 6) {
        etPassword.setError("Password must be at least 6 characters");
        etPassword.requestFocus();
    } else if (!pwd.equals(cpwd)) {
        etConfirm.setError("Passwords do not match");
        etConfirm.requestFocus();
    } else {
        Toast.makeText(this, "Registration successful", Toast.LENGTH_LONG).show();
    }
});
```

3.4 Live (real-time) validation with TextWatcher

Java

```
etEmail.addTextChangedListener(new TextWatcher() {
    @Override
    public void beforeTextChanged(CharSequence s, int start, int count, int after) { }

    @Override
    public void onTextChanged(CharSequence s, int start, int before, int count) { }

    @Override
    public void afterTextChanged(Editable s) {
        if (!Patterns.EMAIL_ADDRESS.matcher(s.toString()).matches()) {
            etEmail.setError("Invalid email");
        } else {
            etEmail.setError(null);
        }
    }
});
```

3.5 Other useful text field operations

Java

```
etName.setText("");                         // clear field
etName.setText("Rahul");                    // set value
etName.setHint("Name");
etName.setEnabled(false);                   // make read-only / disabled
etName.setSelection(etName.getText().length());   // cursor to end
etPassword.setTransformationMethod(
        PasswordTransformationMethod.getInstance());     // hide password
etPassword.setTransformationMethod(
        HideReturnsTransformationMethod.getInstance());  // show password

// Parse numbers safely
try {
    int age = Integer.parseInt(etAge.getText().toString());
} catch (NumberFormatException e) {
    etAge.setError("Enter a number");
}
```

3.6 AutoCompleteTextView

XML

```
<AutoCompleteTextView
    android:id="@+id/actvCity"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Type a city"
    android:completionThreshold="1" />
```

Java

```
String[] cities = {"Kochi", "Kollam", "Kozhikode", "Thrissur", "Thiruvananthapuram"};
ArrayAdapter<String> adapter = new ArrayAdapter<>(this,
        android.R.layout.simple_dropdown_item_1line, cities);
AutoCompleteTextView actv = findViewById(R.id.actvCity);
actv.setAdapter(adapter);
```

Unit 4: Layouts

A layout is a container (ViewGroup) that decides how child views are positioned. Common layouts:

Layout	Use
LinearLayout	Children in a single row (horizontal) or column (vertical)
RelativeLayout	Children placed relative to each other or to the parent
ConstraintLayout	Flexible, flat hierarchy using constraints (default in Android Studio)
FrameLayout	Stack of views, one on top of another (used for fragments)
TableLayout	Rows and columns like an HTML table
GridLayout	Grid with rowCount / columnCount
ScrollView	Makes content scrollable (allows only ONE child)

4.1 LinearLayout

XML: vertical with weights

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <EditText android:layout_width="match_parent"
        android:layout_height="wrap_content" android:hint="Username" />

    <!-- two buttons sharing the width equally -->
    <LinearLayout android:layout_width="match_parent"
        android:layout_height="wrap_content" android:orientation="horizontal">

        <Button android:layout_width="0dp" android:layout_height="wrap_content"
            android:layout_weight="1" android:text="Login" />
        <Button android:layout_width="0dp" android:layout_height="wrap_content"
            android:layout_weight="1" android:text="Cancel" />
    </LinearLayout>
</LinearLayout>
```

Note: For weights, set the width (horizontal) or height (vertical) to **0dp** and give each child a **layout_weight**.

4.2 RelativeLayout

XML

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">

    <TextView android:id="@+id/tvTitle"
        android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:text="Login" android:layout_centerHorizontal="true" />

    <EditText android:id="@+id/etUser"
        android:layout_width="match_parent" android:layout_height="wrap_content"
        android:layout_below="@id/tvTitle" android:hint="Username" />

    <Button android:id="@+id/btnLogin"
        android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:layout_below="@id/etUser"
        android:layout_alignParentRight="true" android:text="Login" />

    <Button android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:layout_toLeftOf="@id/btnLogin"
        android:layout_alignTop="@id/btnLogin" android:text="Cancel" />
</RelativeLayout>
```

Frequently used attributes: `layout_alignParentTop/Bottom/Left/Right`, `layout_centerInParent`, `layout_centerHorizontal/Vertical`, `layout_above`, `layout_below`, `layout_toLeftOf`, `layout_toRightOf`, `layout_alignLeft/Right/Top/Bottom`.

4.3 ConstraintLayout

XML

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView android:id="@+id/tvLabel"
        android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:text="Name"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        android:layout_margin="16dp" />

    <EditText android:id="@+id/etName"
        android:layout_width="0dp" android:layout_height="wrap_content"
        android:hint="Enter name"
        app:layout_constraintStart_toEndOf="@id/tvLabel"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintBaseline_toBaselineOf="@id/tvLabel"
        android:layout_marginStart="8dp" android:layout_marginEnd="16dp" />

    <Button android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:text="Centered"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Note: 0dp in ConstraintLayout means *match constraints*. Every view needs at least one horizontal and one vertical constraint.

4.4 FrameLayout

XML

```
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="200dp">

    <ImageView android:layout_width="match_parent" android:layout_height="match_parent"
        android:src="@drawable/banner" android:scaleType="centerCrop" />

    <TextView android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:layout_gravity="bottom|center_horizontal"
        android:text="Caption over image" android:textColor="#FFFFFF" />
</FrameLayout>
```

4.5 TableLayout

XML

```
<TableLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="wrap_content"
    android:stretchColumns="1">

    <TableRow>
        <TextView android:text="Name:" android:padding="8dp" />
        <EditText android:hint="Enter name" />
    </TableRow>
    <TableRow>
        <TextView android:text="Email:" android:padding="8dp" />
        <EditText android:hint="Enter email" />
    </TableRow>
    <TableRow>
        <Button android:layout_column="1" android:text="Submit" />
    </TableRow>
</TableLayout>
```

4.6 GridLayout

XML: simple 2x2 calculator-style grid

```
<GridLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="wrap_content"
    android:columnCount="2" android:rowCount="2">

    <Button android:layout_width="0dp" android:layout_height="wrap_content"
        android:layout_columnWeight="1" android:text="1" />
    <Button android:layout_width="0dp" android:layout_height="wrap_content"
        android:layout_columnWeight="1" android:text="2" />
    <Button android:layout_width="0dp" android:layout_height="wrap_content"
        android:layout_columnWeight="1" android:text="3" />
    <Button android:layout_width="0dp" android:layout_height="wrap_content"
        android:layout_columnWeight="1" android:text="4" />
</GridLayout>
```

4.7 ScrollView

XML

```
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent">

    <LinearLayout android:layout_width="match_parent"
        android:layout_height="wrap_content" android:orientation="vertical">
        <!-- many views here -->
    </LinearLayout>
</ScrollView>
```

4.8 Creating a layout programmatically (in Java)

Java

```
LinearLayout layout = new LinearLayout(this);
layout.setOrientation(LinearLayout.VERTICAL);
layout.setLayoutParams(new LinearLayout.LayoutParams(
        LinearLayout.LayoutParams.MATCH_PARENT,
        LinearLayout.LayoutParams.MATCH_PARENT));

TextView tv = new TextView(this);
tv.setText("Created in code");
layout.addView(tv);

Button btn = new Button(this);
btn.setText("Dynamic button");
layout.addView(btn);

setContentView(layout);
```

Unit 5: Adapters, Spinner and ListView

An **Adapter** is a bridge between a data source (array, list, database) and an AdapterView (ListView, Spinner, GridView). The adapter converts each data item into a View.

Adapter	Use
ArrayAdapter	Simple arrays / lists of Strings or objects
BaseAdapter	Custom adapter with your own row layout (full control)
SimpleAdapter	Maps data from a List of Maps to views in a row layout
CursorAdapter / SimpleCursorAdapter	Data from SQLite Cursor
RecyclerView.Adapter	Modern, efficient replacement for ListView (advanced)

5.1 Spinner (drop-down list)

XML

```
<Spinner
    android:id="@+id/spCourse"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />
```

Java: Spinner from a Java array

```
Spinner spCourse = findViewById(R.id.spCourse);
String[] courses = {"Select course", "MCA", "MBA", "BCA", "B.Tech"};

ArrayAdapter<String> adapter = new ArrayAdapter<>(this,
        android.R.layout.simple_spinner_item, courses);
adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
spCourse.setAdapter(adapter);

// Item selected listener
spCourse.setOnItemSelectedListener(new AdapterView.OnItemSelectedListener() {
    @Override
    public void onItemSelected(AdapterView<?> parent, View view, int position, long id) {
        String selected = parent.getItemAtPosition(position).toString();
        if (position > 0)
            Toast.makeText(MainActivity.this, selected, Toast.LENGTH_SHORT).show();
    }
    @Override
    public void onNothingSelected(AdapterView<?> parent) { }
});

// Get selected item later
String value = spCourse.getSelectedItem().toString();
int pos = spCourse.getSelectedItemPosition();
spCourse.setSelection(2);          // select "MBA" programmatically
```

Spinner from strings.xml (static data)

```
<!-- res/values/strings.xml -->
<string-array name="course_array">
    <item>MCA</item>
    <item>MBA</item>
    <item>BCA</item>
</string-array>

<!-- activity_main.xml : no Java needed for adapter -->
<Spinner android:id="@+id/spCourse"
    android:layout_width="match_parent" android:layout_height="wrap_content"
    android:entries="@array/course_array" />

<!-- Or in Java -->
ArrayAdapter<CharSequence> ad = ArrayAdapter.createFromResource(this,
        R.array.course_array, android.R.layout.simple_spinner_item);
ad.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
spCourse.setAdapter(ad);
```

5.2 ListView with ArrayAdapter

XML

```
<ListView
    android:id="@+id/listView"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

Java

```
ListView listView = findViewById(R.id.listView);
ArrayList<String> names = new ArrayList<>();
names.add("Anu"); names.add("Binu"); names.add("Chitra"); names.add("Deepak");

ArrayAdapter<String> adapter = new ArrayAdapter<>(this,
        android.R.layout.simple_list_item_1, names);
listView.setAdapter(adapter);

// Click on item
listView.setOnItemClickListener((parent, view, position, id) -> {
    String item = names.get(position);
    Toast.makeText(this, "Clicked: " + item, Toast.LENGTH_SHORT).show();
});

// Long click -> delete item
listView.setOnItemLongClickListener((parent, view, position, id) -> {
    names.remove(position);
    adapter.notifyDataSetChanged();     // refresh list after change
    return true;                        // true = event consumed
});

// Add new item
names.add("Eldho");
adapter.notifyDataSetChanged();
```

5.3 Multiple-choice ListView

Java

```
listView.setChoiceMode(ListView.CHOICE_MODE_MULTIPLE);
ArrayAdapter<String> adapter = new ArrayAdapter<>(this,
        android.R.layout.simple_list_item_multiple_choice, names);
listView.setAdapter(adapter);

// Get all checked items
SparseBooleanArray checked = listView.getCheckedItemPositions();
for (int i = 0; i < names.size(); i++) {
    if (checked.get(i)) Log.d("SEL", names.get(i));
}
```

5.4 Custom ListView using BaseAdapter

Step 1: model class, Step 2: row layout, Step 3: custom adapter, Step 4: set adapter in the Activity.

Student.java (model)

```
public class Student {
    private String name;
    private String course;

    public Student(String name, String course) {
        this.name = name;
        this.course = course;
    }
    public String getName()   { return name; }
    public String getCourse() { return course; }
}
```

res/layout/row_student.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="wrap_content"
    android:orientation="vertical" android:padding="12dp">

    <TextView android:id="@+id/tvName" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:textSize="18sp"
        android:textStyle="bold" />
    <TextView android:id="@+id/tvCourse" android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
</LinearLayout>
```

StudentAdapter.java

```
public class StudentAdapter extends BaseAdapter {
    private Context context;
    private ArrayList<Student> list;

    public StudentAdapter(Context context, ArrayList<Student> list) {
        this.context = context;
        this.list = list;
    }

    @Override public int getCount()           { return list.size(); }
    @Override public Object getItem(int pos)  { return list.get(pos); }
    @Override public long getItemId(int pos)  { return pos; }

    @Override
    public View getView(int position, View convertView, ViewGroup parent) {
        if (convertView == null) {          // reuse old views (performance)
            convertView = LayoutInflater.from(context)
                    .inflate(R.layout.row_student, parent, false);
        }
        TextView tvName   = convertView.findViewById(R.id.tvName);
        TextView tvCourse = convertView.findViewById(R.id.tvCourse);

        Student s = list.get(position);
        tvName.setText(s.getName());
        tvCourse.setText(s.getCourse());
        return convertView;
    }
}
```

MainActivity.java

```
ArrayList<Student> students = new ArrayList<>();
students.add(new Student("Anu", "MCA"));
students.add(new Student("Rahul", "MBA"));

ListView listView = findViewById(R.id.listView);
StudentAdapter adapter = new StudentAdapter(this, students);
listView.setAdapter(adapter);

listView.setOnItemClickListener((parent, view, position, id) ->
    Toast.makeText(this, students.get(position).getName(),
                   Toast.LENGTH_SHORT).show());
```

5.5 Complete example: add items to a ListView from an EditText

Java

```
EditText etItem = findViewById(R.id.etItem);
Button btnAdd = findViewById(R.id.btnAdd);
ListView lv = findViewById(R.id.listView);

ArrayList<String> items = new ArrayList<>();
ArrayAdapter<String> adapter = new ArrayAdapter<>(this,
        android.R.layout.simple_list_item_1, items);
lv.setAdapter(adapter);

btnAdd.setOnClickListener(v -> {
    String text = etItem.getText().toString().trim();
    if (text.isEmpty()) {
        etItem.setError("Enter something");
        return;
    }
    items.add(text);
    adapter.notifyDataSetChanged();
    etItem.setText("");
});
```

5.6 GridView (quick example)

XML + Java

```
<GridView android:id="@+id/gridView" android:layout_width="match_parent"
    android:layout_height="wrap_content" android:numColumns="3"
    android:horizontalSpacing="8dp" android:verticalSpacing="8dp" />

GridView gv = findViewById(R.id.gridView);
String[] data = {"A", "B", "C", "D", "E", "F"};
gv.setAdapter(new ArrayAdapter<>(this, android.R.layout.simple_list_item_1, data));
```

Unit 6: AlertDialog, Toast and Popup

6.1 Toast

A Toast shows a small, temporary message that disappears automatically. It does not take focus.

Java

```
// Basic
Toast.makeText(getApplicationContext(), "Saved!", Toast.LENGTH_SHORT).show();
Toast.makeText(MainActivity.this, "Long message", Toast.LENGTH_LONG).show();

// Change position
Toast toast = Toast.makeText(this, "Top toast", Toast.LENGTH_SHORT);
toast.setGravity(Gravity.TOP | Gravity.CENTER_HORIZONTAL, 0, 100);
toast.show();
```

Custom Toast layout (res/layout/custom_toast.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/toastRoot"
    android:layout_width="wrap_content" android:layout_height="wrap_content"
    android:background="#333333" android:padding="12dp"
    android:orientation="horizontal">

    <ImageView android:layout_width="24dp" android:layout_height="24dp"
        android:src="@android:drawable/ic_dialog_info" />
    <TextView android:id="@+id/toastText" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:layout_marginStart="8dp"
        android:textColor="#FFFFFF" />
</LinearLayout>
```

Java

```
LayoutInflater inflater = getLayoutInflater();
View layout = inflater.inflate(R.layout.custom_toast, findViewById(R.id.toastRoot));
((TextView) layout.findViewById(R.id.toastText)).setText("Custom toast");

Toast toast = new Toast(getApplicationContext());
toast.setDuration(Toast.LENGTH_LONG);
toast.setView(layout);
toast.show();
// Note: custom toast views are ignored on Android 11+ for background apps.
```

Note: Snackbar (Material library) is the modern alternative: `Snackbar.make(view, "Deleted", Snackbar.LENGTH_LONG).setAction("UNDO", v -> {...}).show();`

6.2 AlertDialog

Simple dialog with Yes / No buttons

```
new AlertDialog.Builder(this)
    .setTitle("Exit")
    .setMessage("Do you want to exit the app?")
    .setIcon(android.R.drawable.ic_dialog_alert)
    .setCancelable(false)                          // cannot dismiss by back/outside
    .setPositiveButton("Yes", (dialog, which) -> finish())
    .setNegativeButton("No", (dialog, which) -> dialog.dismiss())
    .setNeutralButton("Later", null)
    .show();
```

Single-choice list dialog

```
String[] colors = {"Red", "Green", "Blue"};
new AlertDialog.Builder(this)
    .setTitle("Choose colour")
    .setSingleChoiceItems(colors, 0, (dialog, which) -> {
        Toast.makeText(this, colors[which], Toast.LENGTH_SHORT).show();
    })
    .setPositiveButton("OK", null)
    .show();
```

Simple items list (click closes dialog)

```
new AlertDialog.Builder(this)
    .setTitle("Pick one")
    .setItems(new String[]{"Camera", "Gallery"}, (dialog, which) -> {
        if (which == 0) { /* open camera */ } else { /* open gallery */ }
    })
    .show();
```

Multi-choice dialog

```
String[] langs = {"Java", "Kotlin", "Python"};
boolean[] checked = {false, false, false};
new AlertDialog.Builder(this)
    .setTitle("Select languages")
    .setMultiChoiceItems(langs, checked, (dialog, which, isChecked) ->
            checked[which] = isChecked)
    .setPositiveButton("Done", (d, w) -> {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < langs.length; i++)
            if (checked[i]) sb.append(langs[i]).append(" ");
        Toast.makeText(this, sb.toString(), Toast.LENGTH_SHORT).show();
    })
    .show();
```

Custom dialog layout with EditText (e.g. login/input)

```
<!-- res/layout/dialog_input.xml -->
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="wrap_content"
    android:orientation="vertical" android:padding="16dp">
    <EditText android:id="@+id/etDialogName" android:layout_width="match_parent"
        android:layout_height="wrap_content" android:hint="Enter name" />
</LinearLayout>

// Java
View view = getLayoutInflater().inflate(R.layout.dialog_input, null);
EditText etDialogName = view.findViewById(R.id.etDialogName);

new AlertDialog.Builder(this)
    .setTitle("Your name")
    .setView(view)
    .setPositiveButton("Save", (d, w) -> {
        String n = etDialogName.getText().toString();
        tvResult.setText("Hi " + n);
    })
    .setNegativeButton("Cancel", null)
    .show();
```

Confirm on back press

```
@Override
public void onBackPressed() {            // deprecated in newer APIs but still works
    new AlertDialog.Builder(this)
        .setMessage("Exit app?")
        .setPositiveButton("Yes", (d, w) -> super.onBackPressed())
        .setNegativeButton("No", null)
        .show();
}
```

6.3 ProgressDialog alternative

`ProgressDialog` is deprecated. Use a `ProgressBar` in XML and toggle its visibility with `setVisibility(View.VISIBLE / View.GONE)`.

6.4 PopupMenu

res/menu/popup_menu.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:id="@+id/menu_edit"   android:title="Edit" />
    <item android:id="@+id/menu_delete" android:title="Delete" />
    <item android:id="@+id/menu_share"  android:title="Share" />
</menu>
```

Java

```
Button btnMenu = findViewById(R.id.btnMenu);
btnMenu.setOnClickListener(v -> {
    PopupMenu popup = new PopupMenu(MainActivity.this, btnMenu);
    popup.getMenuInflater().inflate(R.menu.popup_menu, popup.getMenu());

    popup.setOnMenuItemClickListener(item -> {
        int id = item.getItemId();
        if (id == R.id.menu_edit)        toast("Edit");
        else if (id == R.id.menu_delete) toast("Delete");
        else if (id == R.id.menu_share)  toast("Share");
        return true;
    });
    popup.show();
});
```

6.5 Options Menu (action bar menu) and Context Menu

Java: Options menu

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    getMenuInflater().inflate(R.menu.main_menu, menu);
    return true;
}

@Override
public boolean onOptionsItemSelected(MenuItem item) {
    if (item.getItemId() == R.id.action_settings) {
        Toast.makeText(this, "Settings", Toast.LENGTH_SHORT).show();
        return true;
    }
    return super.onOptionsItemSelected(item);
}
```

Java: Context menu (long-press on a view)

```
registerForContextMenu(listView);

@Override
public void onCreateContextMenu(ContextMenu menu, View v,
                                ContextMenu.ContextMenuInfo info) {
    super.onCreateContextMenu(menu, v, info);
    menu.setHeaderTitle("Choose action");
    menu.add(0, 1, 0, "Edit");
    menu.add(0, 2, 0, "Delete");
}

@Override
public boolean onContextItemSelected(MenuItem item) {
    if (item.getItemId() == 2) { /* delete */ }
    return super.onContextItemSelected(item);
}
```

6.6 PopupWindow (custom floating view)

Java

```
View popupView = getLayoutInflater().inflate(R.layout.popup_layout, null);

PopupWindow popupWindow = new PopupWindow(popupView,
        ViewGroup.LayoutParams.WRAP_CONTENT,
        ViewGroup.LayoutParams.WRAP_CONTENT,
        true);                                  // true = focusable (dismiss on outside tap)

popupWindow.setElevation(10f);

Button btnShow = findViewById(R.id.btnShow);
btnShow.setOnClickListener(v -> popupWindow.showAsDropDown(btnShow, 0, 10));
// or centred on the screen:
// popupWindow.showAtLocation(v, Gravity.CENTER, 0, 0);

Button btnClose = popupView.findViewById(R.id.btnClose);
btnClose.setOnClickListener(v -> popupWindow.dismiss());
```

Unit 7: Fragments

A **Fragment** is a reusable portion of UI with its own layout and lifecycle, hosted inside an Activity. Fragments make it easy to build flexible screens (e.g. tabs, phone vs tablet layouts).

7.1 Fragment lifecycle (order of callbacks)

- **onAttach()** - fragment attached to the activity
- **onCreate()** - initialise non-UI items
- **onCreateView()** - inflate and return the layout
- **onViewCreated() / onActivityCreated()** - view ready; find views, set listeners
- **onStart() / onResume()** - visible and interactive
- **onPause() / onStop()** - leaving screen
- **onDestroyView() / onDestroy() / onDetach()** - cleanup

7.2 Static fragment (declared in XML)

res/layout/fragment_first.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:orientation="vertical" android:padding="16dp">

    <TextView android:id="@+id/tvFragment" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="I am First Fragment" />
    <Button android:id="@+id/btnFrag" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Click" />
</LinearLayout>
```

FirstFragment.java

```
public class FirstFragment extends Fragment {

    public FirstFragment() { /* required empty constructor */ }

    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {
        return inflater.inflate(R.layout.fragment_first, container, false);
    }

    @Override
    public void onViewCreated(View view, Bundle savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);
        Button btn = view.findViewById(R.id.btnFrag);
        btn.setOnClickListener(v ->
            Toast.makeText(getContext(), "Fragment button", Toast.LENGTH_SHORT).show());
    }
}
```

activity_main.xml (static use)

```
<androidx.fragment.app.FragmentContainerView
    android:id="@+id/fragmentContainer"
    android:name="com.example.myapp.FirstFragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

7.3 Dynamic fragment (add / replace in Java)

activity_main.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:orientation="vertical">

    <LinearLayout android:layout_width="match_parent"
        android:layout_height="wrap_content" android:orientation="horizontal">
        <Button android:id="@+id/btnFirst" android:layout_width="0dp"
            android:layout_height="wrap_content" android:layout_weight="1"
            android:text="First" />
        <Button android:id="@+id/btnSecond" android:layout_width="0dp"
            android:layout_height="wrap_content" android:layout_weight="1"
            android:text="Second" />
    </LinearLayout>

    <FrameLayout android:id="@+id/frameContainer"
        android:layout_width="match_parent" android:layout_height="match_parent" />
</LinearLayout>
```

MainActivity.java

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        findViewById(R.id.btnFirst).setOnClickListener(v ->
                loadFragment(new FirstFragment(), false));
        findViewById(R.id.btnSecond).setOnClickListener(v ->
                loadFragment(new SecondFragment(), true));

        if (savedInstanceState == null) {          // load default fragment
            loadFragment(new FirstFragment(), false);
        }
    }

    private void loadFragment(Fragment fragment, boolean addToBackStack) {
        FragmentManager fm = getSupportFragmentManager();
        FragmentTransaction ft = fm.beginTransaction();
        ft.replace(R.id.frameContainer, fragment);     // add() / remove() also exist
        if (addToBackStack) ft.addToBackStack(null);   // back button returns to previous
        ft.commit();
    }
}
```

7.4 Passing data: Activity to Fragment (Bundle / arguments)

Java: send

```
SecondFragment fragment = new SecondFragment();
Bundle bundle = new Bundle();
bundle.putString("name", "Anu");
bundle.putInt("age", 22);
fragment.setArguments(bundle);
// then loadFragment(fragment, true);
```

Java: receive in the fragment

```
@Override
public void onViewCreated(View view, Bundle savedInstanceState) {
    super.onViewCreated(view, savedInstanceState);
    if (getArguments() != null) {
        String name = getArguments().getString("name");
        int age = getArguments().getInt("age", 0);
        TextView tv = view.findViewById(R.id.tvData);
        tv.setText(name + ", " + age);
    }
}
```

7.5 Passing data: Fragment to Activity (interface callback)

Java: fragment

```
public class FormFragment extends Fragment {

    public interface OnDataSent {
        void onDataSent(String data);
    }
    private OnDataSent listener;

    @Override
    public void onAttach(Context context) {
        super.onAttach(context);
        if (context instanceof OnDataSent) listener = (OnDataSent) context;
        else throw new RuntimeException("Activity must implement OnDataSent");
    }

    // call this when the button inside the fragment is clicked
    private void send(String text) { listener.onDataSent(text); }
}
```

Java: activity

```
public class MainActivity extends AppCompatActivity
        implements FormFragment.OnDataSent {

    @Override
    public void onDataSent(String data) {
        Toast.makeText(this, "Got: " + data, Toast.LENGTH_SHORT).show();
    }
}
```

7.6 Fragment to Fragment: Fragment Result API

Java

```
// Sender fragment
Bundle result = new Bundle();
result.putString("key", "Hello from A");
getParentFragmentManager().setFragmentResult("requestKey", result);

// Receiver fragment (in onCreate / onViewCreated)
getParentFragmentManager().setFragmentResultListener("requestKey", this,
        (requestKey, bundle) -> {
            String msg = bundle.getString("key");
            tvData.setText(msg);
        });
```

7.7 Fragment with BottomNavigationView (quick snippet)

XML + Java

```
<com.google.android.material.bottomnavigation.BottomNavigationView
    android:id="@+id/bottomNav" android:layout_width="match_parent"
    android:layout_height="wrap_content" android:layout_gravity="bottom"
    app:menu="@menu/bottom_menu" />

BottomNavigationView nav = findViewById(R.id.bottomNav);
nav.setOnItemSelectedListener(item -> {
    int id = item.getItemId();
    if (id == R.id.nav_home)      loadFragment(new HomeFragment(), false);
    else if (id == R.id.nav_profile) loadFragment(new ProfileFragment(), false);
    return true;
});
```

Unit 8: IME (Keyboard), Date and Time

IME = Input Method Editor, i.e. the on-screen keyboard. The pickers below let users choose date and time easily.

8.1 IME options (action button on the keyboard)

XML

```
<EditText
    android:id="@+id/etSearch"
    android:layout_width="match_parent" android:layout_height="wrap_content"
    android:hint="Search..."
    android:inputType="text"
    android:imeOptions="actionSearch"
    android:maxLines="1" />
<!-- other values: actionDone, actionNext, actionGo, actionSend, actionPrevious -->
```

Java: handle IME action

```
etSearch.setOnEditorActionListener((v, actionId, event) -> {
    if (actionId == EditorInfo.IME_ACTION_SEARCH) {
        String q = etSearch.getText().toString();
        Toast.makeText(this, "Searching: " + q, Toast.LENGTH_SHORT).show();
        return true;
    }
    return false;
});
```

Java: show / hide the soft keyboard

```
// Hide
InputMethodManager imm =
        (InputMethodManager) getSystemService(Context.INPUT_METHOD_SERVICE);
imm.hideSoftInputFromWindow(etSearch.getWindowToken(), 0);

// Show
etSearch.requestFocus();
imm.showSoftInput(etSearch, InputMethodManager.SHOW_IMPLICIT);
```

AndroidManifest.xml: keyboard behaviour for an activity

```
<activity android:name=".MainActivity"
    android:windowSoftInputMode="adjustResize|stateHidden" />
<!-- adjustResize : layout shrinks so input stays visible
     adjustPan    : screen pans up
     stateHidden  : keyboard hidden when activity opens -->
```

8.2 DatePickerDialog

XML

```
<EditText android:id="@+id/etDate" android:layout_width="match_parent"
    android:layout_height="wrap_content" android:hint="Select date"
    android:focusable="false" android:clickable="true" />
```

Java

```
EditText etDate = findViewById(R.id.etDate);

etDate.setOnClickListener(v -> {
    Calendar cal = Calendar.getInstance();
    int year  = cal.get(Calendar.YEAR);
    int month = cal.get(Calendar.MONTH);          // 0-based (0 = January)
    int day   = cal.get(Calendar.DAY_OF_MONTH);

    DatePickerDialog dialog = new DatePickerDialog(MainActivity.this,
        (view, y, m, d) -> etDate.setText(d + "/" + (m + 1) + "/" + y),
        year, month, day);

    dialog.getDatePicker().setMaxDate(System.currentTimeMillis());  // no future dates
    dialog.show();
});
```

8.3 TimePickerDialog

Java

```
EditText etTime = findViewById(R.id.etTime);

etTime.setOnClickListener(v -> {
    Calendar cal = Calendar.getInstance();
    int hour   = cal.get(Calendar.HOUR_OF_DAY);
    int minute = cal.get(Calendar.MINUTE);

    TimePickerDialog dialog = new TimePickerDialog(MainActivity.this,
        (view, h, m) -> {
            String amPm = (h >= 12) ? "PM" : "AM";
            int h12 = (h % 12 == 0) ? 12 : h % 12;
            etTime.setText(String.format(Locale.getDefault(),
                    "%02d:%02d %s", h12, m, amPm));
        },
        hour, minute, false);       // false = 12-hour format, true = 24-hour
    dialog.show();
});
```

8.4 DatePicker and TimePicker widgets in XML

XML + Java

```
<DatePicker android:id="@+id/datePicker" android:layout_width="wrap_content"
    android:layout_height="wrap_content" android:datePickerMode="spinner"
    android:calendarViewShown="false" />

<TimePicker android:id="@+id/timePicker" android:layout_width="wrap_content"
    android:layout_height="wrap_content" android:timePickerMode="spinner" />

DatePicker dp = findViewById(R.id.datePicker);
String date = dp.getDayOfMonth() + "/" + (dp.getMonth() + 1) + "/" + dp.getYear();

TimePicker tp = findViewById(R.id.timePicker);
int h = tp.getHour();        // API 23+
int m = tp.getMinute();
```

8.5 Current date and time, formatting

Java

```
SimpleDateFormat df = new SimpleDateFormat("dd-MM-yyyy", Locale.getDefault());
String today = df.format(new Date());

SimpleDateFormat tf = new SimpleDateFormat("hh:mm a", Locale.getDefault());
String now = tf.format(Calendar.getInstance().getTime());

// Show in a TextView
tvDate.setText("Today: " + today + "  Time: " + now);
```

8.6 Calculate age from the selected date of birth

Java

```
// inside the DatePickerDialog callback: (view, y, m, d)
Calendar today = Calendar.getInstance();
int age = today.get(Calendar.YEAR) - y;
if (today.get(Calendar.MONTH) < m ||
   (today.get(Calendar.MONTH) == m && today.get(Calendar.DAY_OF_MONTH) < d)) {
    age--;
}
tvAge.setText("Age: " + age);
```

8.7 TextClock and AnalogClock (display only)

XML

```
<TextClock android:layout_width="wrap_content" android:layout_height="wrap_content"
    android:format12Hour="hh:mm:ss a" android:textSize="28sp" />
```

Unit 9: SharedPreferences

SharedPreferences stores small amounts of primitive data (String, int, boolean, float, long, Set<String>) as key-value pairs in an XML file inside the app's private storage. Used for settings, login status, remember-me, simple scores.

Method	Description
getSharedPreferences(name, mode)	Open / create a named preferences file (use MODE_PRIVATE)
getPreferences(mode)	Preferences file private to the activity
edit()	Returns SharedPreferences.Editor
putString / putInt / putBoolean...	Write values (call on Editor)
apply()	Save asynchronously (preferred)
commit()	Save synchronously, returns true/false
getString(key, default) ...	Read value, or default if key is absent
remove(key) / clear()	Delete one key / everything
contains(key)	Check if key exists

9.1 Saving data

Java

```
SharedPreferences sp = getSharedPreferences("MyPrefs", MODE_PRIVATE);
SharedPreferences.Editor editor = sp.edit();

editor.putString("username", "anu");
editor.putInt("age", 22);
editor.putBoolean("isLoggedIn", true);
editor.putFloat("cgpa", 8.5f);
editor.putLong("lastLogin", System.currentTimeMillis());
editor.apply();                  // or editor.commit();
```

9.2 Reading data

Java

```
SharedPreferences sp = getSharedPreferences("MyPrefs", MODE_PRIVATE);

String user  = sp.getString("username", "");        // "" = default value
int age      = sp.getInt("age", 0);
boolean flag = sp.getBoolean("isLoggedIn", false);
float cgpa   = sp.getFloat("cgpa", 0f);

if (sp.contains("username")) { /* key exists */ }
```

9.3 Removing / clearing data

Java

```
SharedPreferences.Editor editor = getSharedPreferences("MyPrefs", MODE_PRIVATE).edit();
editor.remove("username");       // remove a single key
editor.clear();                  // remove all keys
editor.apply();
```

9.4 Complete example: Save / Load / Clear form data

activity_main.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:orientation="vertical" android:padding="16dp">

    <EditText android:id="@+id/etName" android:layout_width="match_parent"
        android:layout_height="wrap_content" android:hint="Name" />
    <EditText android:id="@+id/etEmail" android:layout_width="match_parent"
        android:layout_height="wrap_content" android:hint="Email"
        android:inputType="textEmailAddress" />
    <CheckBox android:id="@+id/cbRemember" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Remember me" />

    <Button android:id="@+id/btnSave" android:layout_width="match_parent"
        android:layout_height="wrap_content" android:text="Save" />
    <Button android:id="@+id/btnLoad" android:layout_width="match_parent"
        android:layout_height="wrap_content" android:text="Load" />
    <Button android:id="@+id/btnClear" android:layout_width="match_parent"
        android:layout_height="wrap_content" android:text="Clear" />
    <TextView android:id="@+id/tvShow" android:layout_width="match_parent"
        android:layout_height="wrap_content" android:paddingTop="12dp" />
</LinearLayout>
```

MainActivity.java

```
public class MainActivity extends AppCompatActivity {
    private static final String PREF_NAME = "UserData";
    EditText etName, etEmail; CheckBox cbRemember; TextView tvShow;
    SharedPreferences sp;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        etName = findViewById(R.id.etName);
        etEmail = findViewById(R.id.etEmail);
        cbRemember = findViewById(R.id.cbRemember);
        tvShow = findViewById(R.id.tvShow);
        sp = getSharedPreferences(PREF_NAME, MODE_PRIVATE);

        findViewById(R.id.btnSave).setOnClickListener(v -> {
            SharedPreferences.Editor ed = sp.edit();
            ed.putString("name", etName.getText().toString());
            ed.putString("email", etEmail.getText().toString());
            ed.putBoolean("remember", cbRemember.isChecked());
            ed.apply();
            Toast.makeText(this, "Saved", Toast.LENGTH_SHORT).show();
        });

        findViewById(R.id.btnLoad).setOnClickListener(v -> {
            String n = sp.getString("name", "Not saved");
            String e = sp.getString("email", "Not saved");
            tvShow.setText("Name: " + n + "\nEmail: " + e);
        });

        findViewById(R.id.btnClear).setOnClickListener(v -> {
            sp.edit().clear().apply();
            tvShow.setText("");
        });

        // Auto-fill if "remember me" was ticked
        if (sp.getBoolean("remember", false)) {
            etName.setText(sp.getString("name", ""));
            etEmail.setText(sp.getString("email", ""));
            cbRemember.setChecked(true);
        }
    }
}
```

9.5 Login session example (skip login screen if already logged in)

LoginActivity.java

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);

    SharedPreferences sp = getSharedPreferences("Session", MODE_PRIVATE);
    if (sp.getBoolean("isLoggedIn", false)) {
        startActivity(new Intent(this, HomeActivity.class));
        finish();                       // remove login screen from back stack
        return;
    }
    setContentView(R.layout.activity_login);

    btnLogin.setOnClickListener(v -> {
        String u = etUser.getText().toString();
        String p = etPass.getText().toString();
        if (u.equals("admin") && p.equals("1234")) {      // demo only
            sp.edit().putBoolean("isLoggedIn", true)
                     .putString("user", u).apply();
            startActivity(new Intent(this, HomeActivity.class));
            finish();
        } else {
            Toast.makeText(this, "Invalid credentials", Toast.LENGTH_SHORT).show();
        }
    });
}
```

HomeActivity.java: logout

```
btnLogout.setOnClickListener(v -> {
    getSharedPreferences("Session", MODE_PRIVATE).edit().clear().apply();
    Intent i = new Intent(this, LoginActivity.class);
    i.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TASK);
    startActivity(i);
});
```

> **Note:** Never store real passwords in plain SharedPreferences. For sensitive data use EncryptedSharedPreferences or Android Keystore. Also, for larger/structured data use SQLite / Room instead.

9.6 Saving a String Set (list of values)

Java

```
Set<String> hobbies = new HashSet<>();
hobbies.add("Reading"); hobbies.add("Cricket");
sp.edit().putStringSet("hobbies", hobbies).apply();

Set<String> saved = sp.getStringSet("hobbies", new HashSet<>());
```

9.7 Where is the file stored?

`/data/data/<package-name>/shared_prefs/UserData.xml` (visible via Device File Explorer in Android Studio for debuggable apps).

Unit 10: Intents (Explicit and Implicit)

An **Intent** is a messaging object used to request an action from another app component (Activity, Service, Broadcast Receiver).

	Explicit Intent	Implicit Intent
Target	Exact component class is named	Only the action/data is described; system finds a suitable app
Typical use	Open another activity in your own app	Open browser, dial, send email, share, camera, map
Needs intent-filter?	No	Receiving app must declare an intent-filter
Example	new Intent(this, SecondActivity.class)	new Intent(Intent.ACTION_VIEW, uri)

10.1 Explicit intent: open another Activity

Java

```
Intent intent = new Intent(MainActivity.this, SecondActivity.class);
startActivity(intent);

// Close current activity as well
finish();
```

AndroidManifest.xml: every activity must be declared

```
<application ...>
    <activity android:name=".MainActivity" android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>

    <activity android:name=".SecondActivity" android:exported="false" />
</application>
```

Note: Android Studio: *File* > *New* > *Activity* creates the Java class, layout and manifest entry automatically.

10.2 Passing data with putExtra()

Sender: MainActivity.java

```
btnSend.setOnClickListener(v -> {
    String name = etName.getText().toString();
    int age = Integer.parseInt(etAge.getText().toString());

    Intent intent = new Intent(MainActivity.this, SecondActivity.class);
    intent.putExtra("NAME", name);
    intent.putExtra("AGE", age);
    intent.putExtra("PASSED", true);
    startActivity(intent);
});
```

Receiver: SecondActivity.java

```
Intent intent = getIntent();
String name = intent.getStringExtra("NAME");
int age = intent.getIntExtra("AGE", 0);               // 0 = default
boolean passed = intent.getBooleanExtra("PASSED", false);

TextView tv = findViewById(R.id.tvDetails);
tv.setText("Name: " + name + "\nAge: " + age);
```

Using a Bundle

```
Bundle b = new Bundle();
b.putString("city", "Kochi");
b.putInt("pin", 682001);
intent.putExtras(b);

// receiver
Bundle got = getIntent().getExtras();
if (got != null) {
    String city = got.getString("city");
    int pin = got.getInt("pin");
}
```

10.3 Passing an object (Serializable)

Student.java

```
public class Student implements Serializable {
    private String name;
    private int age;
    public Student(String name, int age) { this.name = name; this.age = age; }
    public String getName() { return name; }
    public int getAge()     { return age; }
}
```

Send and receive

```
// sender
Student s = new Student("Anu", 22);
Intent i = new Intent(this, SecondActivity.class);
i.putExtra("student", s);
startActivity(i);

// receiver
Student st = (Student) getIntent().getSerializableExtra("student");
tv.setText(st.getName() + " - " + st.getAge());
// Parcelable is faster than Serializable (recommended for production).
```

10.4 Getting a result back (Activity Result API)

`startActivityForResult()` is deprecated. Use `registerForActivityResult()`.

MainActivity.java (caller)

```
ActivityResultLauncher<Intent> launcher = registerForActivityResult(
    new ActivityResultContracts.StartActivityForResult(),
    result -> {
        if (result.getResultCode() == RESULT_OK && result.getData() != null) {
            String value = result.getData().getStringExtra("RESULT");
            tvResult.setText("Returned: " + value);
        }
    });

btnOpen.setOnClickListener(v ->
    launcher.launch(new Intent(MainActivity.this, SecondActivity.class)));
```

SecondActivity.java (returns data)

```
btnDone.setOnClickListener(v -> {
    Intent data = new Intent();
    data.putExtra("RESULT", etReply.getText().toString());
    setResult(RESULT_OK, data);
    finish();                       // go back to MainActivity
});
```

10.5 Implicit intents (common actions)

Open a web page

```
Intent i = new Intent(Intent.ACTION_VIEW, Uri.parse("https://www.google.com"));
startActivity(i);
```

Dial a number (no permission needed)

```
Intent i = new Intent(Intent.ACTION_DIAL);
i.setData(Uri.parse("tel:9876543210"));
startActivity(i);
```

Make a direct call (needs CALL_PHONE permission)

```
<!-- Manifest -->
<uses-permission android:name="android.permission.CALL_PHONE" />

// Java
if (ContextCompat.checkSelfPermission(this, Manifest.permission.CALL_PHONE)
        != PackageManager.PERMISSION_GRANTED) {
    ActivityCompat.requestPermissions(this,
            new String[]{Manifest.permission.CALL_PHONE}, 101);
} else {
    Intent i = new Intent(Intent.ACTION_CALL, Uri.parse("tel:9876543210"));
    startActivity(i);
}

@Override
public void onRequestPermissionsResult(int code, String[] perms, int[] results) {
    super.onRequestPermissionsResult(code, perms, results);
    if (code == 101 && results.length > 0
            && results[0] == PackageManager.PERMISSION_GRANTED) {
        startActivity(new Intent(Intent.ACTION_CALL, Uri.parse("tel:9876543210")));
    }
}
```

Send an email

```
Intent i = new Intent(Intent.ACTION_SENDTO);
i.setData(Uri.parse("mailto:"));                        // only email apps
i.putExtra(Intent.EXTRA_EMAIL, new String[]{"abc@example.com"});
i.putExtra(Intent.EXTRA_SUBJECT, "Assignment");
i.putExtra(Intent.EXTRA_TEXT, "Please find the details...");
startActivity(Intent.createChooser(i, "Send email using"));
```

Send an SMS (opens messaging app)

```
Intent i = new Intent(Intent.ACTION_SENDTO, Uri.parse("smsto:9876543210"));
i.putExtra("sms_body", "Hello from my app");
startActivity(i);
```

Share text with other apps

```
Intent i = new Intent(Intent.ACTION_SEND);
i.setType("text/plain");
i.putExtra(Intent.EXTRA_SUBJECT, "Check this out");
i.putExtra(Intent.EXTRA_TEXT, "Learn Android at example.com");
startActivity(Intent.createChooser(i, "Share via"));
```

Open a location in maps

```
Uri geo = Uri.parse("geo:9.9312,76.2673?q=Kochi");
Intent i = new Intent(Intent.ACTION_VIEW, geo);
startActivity(i);
```

Open camera / gallery

```
// Camera (thumbnail)
ActivityResultLauncher<Intent> cameraLauncher = registerForActivityResult(
    new ActivityResultContracts.StartActivityForResult(), result -> {
        if (result.getResultCode() == RESULT_OK && result.getData() != null) {
            Bitmap photo = (Bitmap) result.getData().getExtras().get("data");
            imageView.setImageBitmap(photo);
        }
    });
cameraLauncher.launch(new Intent(MediaStore.ACTION_IMAGE_CAPTURE));

// Pick image from gallery
Intent pick = new Intent(Intent.ACTION_PICK, MediaStore.Images.Media.EXTERNAL_CONTENT_URI);
galleryLauncher.launch(pick);       // result.getData().getData() gives image Uri
```

Open app settings / Wi-Fi settings

```
startActivity(new Intent(Settings.ACTION_WIFI_SETTINGS));
startActivity(new Intent(Settings.ACTION_SETTINGS));
```

Set an alarm / open a contact (examples)

```
Intent alarm = new Intent(AlarmClock.ACTION_SET_ALARM);
alarm.putExtra(AlarmClock.EXTRA_HOUR, 7);
alarm.putExtra(AlarmClock.EXTRA_MINUTES, 30);
alarm.putExtra(AlarmClock.EXTRA_MESSAGE, "Wake up");
startActivity(alarm);
```

10.6 Safe launching: check that an app can handle the intent

Java

```
Intent i = new Intent(Intent.ACTION_VIEW, Uri.parse("https://example.com"));
if (i.resolveActivity(getPackageManager()) != null) {
    startActivity(i);
} else {
    Toast.makeText(this, "No app can handle this", Toast.LENGTH_SHORT).show();
}
// Android 11+ : declare <queries> in the manifest for package visibility
//   <queries><intent><action android:name="android.intent.action.VIEW" />
//   <data android:scheme="https" /></intent></queries>
```

10.7 Receiving an implicit intent in your own activity (intent-filter)

AndroidManifest.xml

```
<activity android:name=".ShareReceiverActivity" android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.SEND" />
        <category android:name="android.intent.category.DEFAULT" />
        <data android:mimeType="text/plain" />
    </intent-filter>
</activity>
```

ShareReceiverActivity.java

```
Intent intent = getIntent();
if (Intent.ACTION_SEND.equals(intent.getAction())) {
    String text = intent.getStringExtra(Intent.EXTRA_TEXT);
    tvShared.setText(text);
}
```

10.8 Complete example: Login -> Welcome (explicit intent with data)

MainActivity.java

```
btnLogin.setOnClickListener(v -> {
    String user = etUser.getText().toString().trim();
    String pass = etPass.getText().toString().trim();

    if (user.isEmpty()) { etUser.setError("Required"); return; }
    if (pass.length() < 4) { etPass.setError("Min 4 characters"); return; }

    Intent i = new Intent(MainActivity.this, WelcomeActivity.class);
    i.putExtra("user", user);
    startActivity(i);
});
```

WelcomeActivity.java

```
public class WelcomeActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_welcome);

        String user = getIntent().getStringExtra("user");
        ((TextView) findViewById(R.id.tvWelcome)).setText("Welcome, " + user + "!");

        findViewById(R.id.btnBack).setOnClickListener(v -> finish());
    }
}
```

Unit 11: Quick Revision

11.1 Activity lifecycle

Method	When it is called
onCreate()	Activity first created: setContentView, findViewById, initialise
onStart()	Activity becoming visible
onResume()	Activity in foreground, user can interact
onPause()	Another activity partially covers it; save light data
onStop()	Activity no longer visible
onRestart()	Stopped activity is coming back
onDestroy()	Activity finishing or being destroyed

Logging lifecycle for practice

```
private static final String TAG = "LIFECYCLE";

@Override protected void onStart()   { super.onStart();   Log.d(TAG, "onStart"); }
@Override protected void onResume()  { super.onResume();  Log.d(TAG, "onResume"); }
@Override protected void onPause()   { super.onPause();   Log.d(TAG, "onPause"); }
@Override protected void onStop()    { super.onStop();    Log.d(TAG, "onStop"); }
@Override protected void onDestroy() { super.onDestroy(); Log.d(TAG, "onDestroy"); }
```

11.2 Commonly needed imports

Java

```
import android.os.Bundle;
import android.content.Intent;
import android.content.Context;
import android.content.SharedPreferences;
import android.net.Uri;
import android.view.View;
import android.view.LayoutInflater;
import android.view.ViewGroup;
import android.widget.*;                    // Button, EditText, TextView, Toast ...
import androidx.appcompat.app.AppCompatActivity;
import androidx.appcompat.app.AlertDialog;
import androidx.fragment.app.Fragment;
import androidx.fragment.app.FragmentManager;
import androidx.fragment.app.FragmentTransaction;
import android.app.DatePickerDialog;
import android.app.TimePickerDialog;
import java.util.ArrayList;
import java.util.Calendar;
```

11.3 Cheat sheet: widget to value

Widget	Get value	Set value
EditText	et.getText().toString()	et.setText("..")
TextView	tv.getText().toString()	tv.setText("..")
CheckBox / Switch	cb.isChecked()	cb.setChecked(true)
RadioGroup	rg.getCheckedRadioButtonId()	rg.check(R.id.rbMale)
Spinner	sp.getSelectedItem().toString()	sp.setSelection(pos)
RatingBar	rb.getRating()	rb.setRating(3.5f)
SeekBar / ProgressBar	sb.getProgress()	sb.setProgress(40)
ListView item	adapter.getItem(pos)	list.add(..); adapter.notifyDataSetChanged()

11.4 Frequently asked viva / exam questions

- **Difference between dp and sp?** dp (density-independent pixel) for sizes of views; sp (scale-independent pixel) for text, also scales with user font size.
- **Difference between match_parent and wrap_content?** match_parent fills the parent; wrap_content is just big enough for the content.
- **What is R.java / R class?** Auto-generated class holding integer ids of all resources (layouts, ids, strings, drawables).
- **What is the role of AndroidManifest.xml?** Declares components, permissions, hardware features and the launcher activity.
- **Explain Adapter.** A bridge between data source and AdapterView; its getView() creates the row view for each item.
- **Why notifyDataSetChanged()?** Tells the adapter that the underlying data changed so the list refreshes.
- **Toast vs AlertDialog?** Toast is a passive, auto-dismissing message with no buttons; AlertDialog needs user interaction and can have buttons/lists.
- **apply() vs commit()?** apply() writes asynchronously and returns nothing; commit() writes synchronously and returns a boolean.
- **Explicit vs implicit intent?** Explicit names the target component; implicit declares an action and the system chooses the app.
- **What is a Fragment?** A modular UI portion with its own lifecycle hosted inside an Activity; can be added, replaced or removed at runtime.
- **What is the back stack?** Stack of activities (or fragment transactions) so the Back button returns to the previous screen.
- **visibility: invisible vs gone?** invisible hides but keeps its space; gone hides and removes the space.

11.5 Common errors and fixes

Problem	Likely cause / fix
NullPointerException on a view	findViewById called before setContentView, or wrong id / wrong layout file
ActivityNotFoundException	Activity not declared in AndroidManifest.xml
App crashes on NumberFormatException	Parsing an empty/invalid EditText string; validate before Integer.parseInt()
List does not update	Forgot notifyDataSetChanged() or modified a different list than the adapter holds
Toast not showing	Missing .show() at the end
Permission denied	Add in manifest and request at runtime for dangerous permissions
Layout looks cut on small screens	Wrap content in ScrollView; avoid fixed large dp widths
Fragment crash on rotation	Fragment needs an empty public constructor; pass data through setArguments()

Note: Practice tip: type every example yourself, run it on the emulator, and then modify it (change ids, add one more field, add validation). Understanding by modifying is the best way to prepare for lab exams.